

C notes

1. Structures in C

Definition: A structure is a user-defined data type that groups different data types under one name. **Use case:** Useful for modeling real-world entities (e.g., Student with roll_no, name, age, marks). **Key point:** Unlike arrays, structures can hold heterogeneous data.

Syntax:

```
struct Student {  
    int roll_no;  
    char name[50];  
    float marks;  
};
```

2. Enums in C

Definition: An enum is a user-defined type consisting of named integer constants. **Use case:** Improves readability by replacing raw numbers with meaningful names. **Key point:** Values start at 0 by default unless explicitly set.

Syntax:

```
enum weekday {sunday, monday, tuesday, wednesday};
```

3. Strings in C

Definition: Strings are arrays of characters terminated by a null character \0.

Use case: Used for text manipulation in programs. **Key point:** Cannot assign directly after declaration; must use functions like strcpy(). **Common functions:** strlen(), strcpy(), strcat(), strcmp().

Syntax:

```
char name[] = "Hello";
```

4. Pointers in C

Definition: A pointer is a variable that stores the memory address of another variable. **Use case:** Efficient array traversal, dynamic memory management, function arguments. **Key point:** Dereferencing (*ptr) retrieves the value at the address. **Syntax:**

```
int data = 10;

int *ptr = &data;

printf("%d", *ptr); // prints 10
```

Coding Round Questions

- Employee struct → Create a struct Employee with id, name, salary and print details.
- Enum months → Define an enum for months and print the integer value of March.
- String length → Write a program to input a string and find its length using strlen().
- Pointer basics → Demonstrate pointer usage by printing the value and address of a variable.
- Array traversal → Traverse an array using pointers instead of indices.
- String comparison → Compare two strings using strcmp() and print whether they are equal

```
#include <stdio.h>
#include <string.h>

struct Student{
    int roll_no;
    char name[50];
    int age;
    float marks;
};

enum weekday {sunday, monday, tuesday, wednesday};
```

```
int main(){

    enum weekday today = tuesday;

    printf("%d\n", today);

    struct Student s1;
    s1.roll_no = 10;
    strcpy(s1.name , "jayaprakash");
    // s1.name ="Jayaprakash"; // doesn't work
    s1.age=18;
    s1.marks=430;

    printf("%s\n", s1.name);
    printf("%d\n", s1.roll_no);

    printf("%d\n", s1.age);

    printf("%f\n", s1.marks);


    // 1st way
    char name[]="Hello FIIT";
    // 2nd way
    char str[] = {'H', 'E', 'L', 'L', 'O'};

    //3rd way
    char *strr = "Hello";


    // printf("%s", name);

    // char names[50];

    // scanf(" %s", names);

    // printf("%s\n", names);

    // printf("Length %d\n", strlen(names));

    // char dest[50];

    // strcpy(dest, names);
```

```
// printf("%s\n", dest);

// char fiit[50]="Hello";
// char fiiit[50]="Hello";

// strcat(fiit, fiiit);

// printf("%s\n", fiit);

// printf("Comparation -> %d", strcmp(fiit, fiiit));

// pointer
// int data = 10;

// int *ptr = &data;

// printf("%d", data);

// // dereferencing
// printf("%d", *ptr);

int arr[]={1,2,3,4,5};

int *ptrr = arr;

printf("First elements %d", *ptrr);

ptrr++;

printf("Second elements %d", *ptrr);

// structures
// user-defined data types
return 0;
}
```